

UNIT III PROBLEM SOLVING BASICS

SYLLABUS

Problem Definition-Logical Reasoning-Decomposition-Software Design Concepts-Algorithm: Concept, of an Algorithm, Algorithm Representation, Algorithm Discovery-Algorithmic Structures: Iterative Structures, Recursive Structures- Efficiency and Correctness of Algorithms-Implementation of Algorithms- Fundamental Algorithms: Exchanging values of two variables, Counting, Summation of a set of numbers, Factorial computation, Generation of Fibonacci sequence, Reversing the digits of an integer, Base Conversion

3.1 PROBLEM DEFINITION

A problem in computing is a task that requires a solution through logical reasoning, algorithm development, and implementation. Defining the problem properly ensures clarity in objectives, resources, and expected results.

- In programming, a problem is expressed as:
 - **Input(s):** Information provided to the program.
 - **Process:** Logical steps that transform input into output.
 - **Output(s):** Final result produced.
- Poorly defined problems → wasted time and incorrect solutions.

Step-by-Step Method

1. Identify the **real-world challenge**.
2. Analyze requirements → Inputs / Outputs.
3. Define **scope and constraints**.
4. Establish **success criteria** (how correctness will be measured).
5. Break into smaller tasks.

Illustration Example

Problem: Find the largest number among three inputs.

- **Input:** Three numbers (a, b, c).
- **Output:** Largest number.
- **Process:** Compare numbers using conditional logic.

Pseudocode

Algorithm LargestOfThree

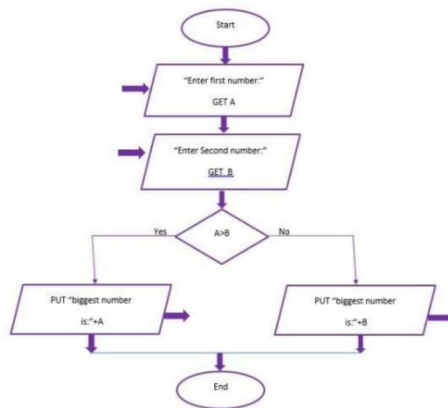
1. Input a, b, c
2. If $a > b$ and $a > c$ then
 largest = a
Else if $b > c$ then
 largest = b
Else

largest = c

3. Print largest

End Algorithm

Flowchart Description



Program

```
#include <stdio.h>
```

```
int main() {
```

```
    int a, b, c, largest;
```

```
    printf("Enter first number: ");
```

```
scanf("%d", &a);

printf("Enter second number: ");

scanf("%d", &b);

printf("Enter third number: ");

scanf("%d", &c);

// Assume first number is largest

largest = a;

// Compare with second

if (b > largest) {

    largest = b;

}

// Compare with third

if (c > largest) {

    largest = c;

}

printf("Largest number is: %d\n", largest);
```

```
return 0;
```

```
}
```

Applications & Case Studies

Applications:

- ✓ Business: Find max profit among branches.
- ✓ Engineering: Max stress/strain values.
- ✓ Sports: Highest score from players.

Case Study: In election result systems, identifying the candidate with maximum votes uses similar logic.

Comparisons / Notes / Tips

- ✓ Good problem definition saves **50% debugging time** later.
- ✓ Avoid vague statements like “find output” – specify exact input/output.
- ✓ Problem definition in programming = **Requirement Analysis** in software development.

3.2 LOGICAL REASONING

Logical reasoning is the ability to use structured thought processes to arrive at conclusions. In computing, it is used to:

- Design algorithms.
- Debug errors.
- Ensure correctness of solutions.

It involves **deductive reasoning** (general $\rightarrow$ specific) and **inductive reasoning** (specific $\rightarrow$ general).

Step-by-Step Method

1. Identify problem conditions.
2. Use rules to connect conditions.
3. Derive logical conclusions.
4. Apply decision-making in algorithms.

Example

Problem: Check if a year is a leap year.

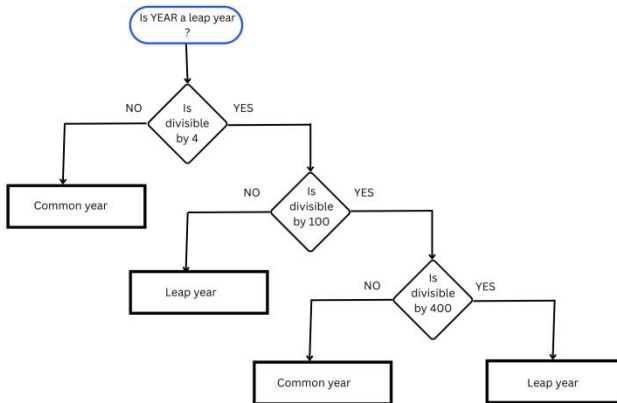
Rule: Year divisible by 4 is leap, but divisible by 100 is not, unless divisible by 400.

Pseudocode

Algorithm LeapYear

```
1. Input year
2. If (year % 400 == 0) then
    print "Leap Year"
   Else if (year % 100 == 0) then
    print "Not Leap Year"
   Else if (year % 4 == 0) then
    print "Leap Year"
   Else
    print "Not Leap Year"
End Algorithm
```

Flowchart Description



Program

```
#include <stdio.h>
```

```
int main() {
```

```
    int year;
```

```
    printf("Enter a year: ");
```

```
    scanf("%d", &year);
```

```
    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))  
    {
```

```
    printf("Leap Year\n");  
  
    } else {  
  
        printf("Not a Leap Year\n");  
  
    }  
  
    return 0;  
  
}
```

Applications & Case Studies

Applications:

- ✓ AI → Logical inference engines.
- ✓ Robotics → Conditional navigation.
- ✓ Banking → Transaction validation.

Case Study: Fraud detection uses logical rules (e.g., impossible location/time combinations).

Notes / Tips

- Logical reasoning = backbone of **if-else** and **loops** in coding.
- Always check all possible conditions (avoid missing cases).

3.3 DECOMPOSITION

Decomposition is the process of **breaking down a complex problem into smaller, manageable sub-problems**. This is a fundamental principle in both **computer science** and **engineering problem solving**. Instead of trying to solve a large problem in one go, we divide it into modules or components that can be solved independently and then combined to form the complete solution.

- Also called “**divide and conquer**”.
- Leads to **modularity, reusability, and simpler debugging**.
- Applied in **algorithms, software design, data structures**, and even **real-world engineering projects**.

Step-by-Step Method of Decomposition

1. **Understand the main problem** – Identify the ultimate goal.
2. **Identify major tasks** – Break the main problem into core functional parts.
3. **Subdivide further** – Keep breaking down until each subtask is simple enough.
4. **Define interactions** – Determine how subtasks communicate or depend on each other.
5. **Implement independently** – Solve/test subtasks separately.
6. **Integrate** – Combine the subtasks to solve the main problem.

Illustration / Example

Problem: Design a program to manage a student's marks and calculate their grade.

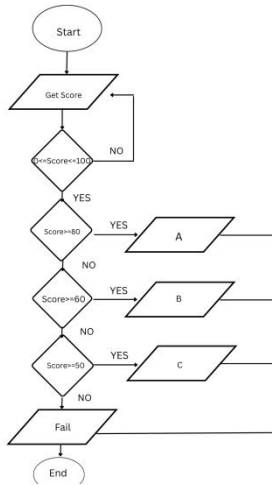
- **Main Problem:** Compute final grade of student.
- **Decomposition:**
 - Input student details.
 - Input subject marks.
 - Compute total.
 - Compute average.
 - Assign grade.
 - Print result.

Pseudocode

Algorithm StudentGrade

1. Input name, roll_no
 2. Input marks for 5 subjects
 3. total = sum(marks)
 4. avg = total / 5
 5. if avg >= 90 then grade = 'A'
 6. else if avg >= 75 then grade = 'B'
 7. else if avg >= 50 then grade = 'C'
 8. else grade = 'F'
 9. Print name, roll_no, total, avg, grade
- End Algorithm

Flowchart Description



Program

```
#include <stdio.h>
```

```
int main() {
```

```
    char name[50], roll[20], grade;
```

```
    int i, mark, total = 0;
```

```
    float avg;
```

```
    printf("Enter Student Name: ");
```

```
    scanf("%s", name);
```

```
    printf("Enter Roll Number: ");
```

```
scanf("%s", roll);

for(i = 1; i <= 5; i++) {

    printf("Enter marks for subject %d: ", i);

    scanf("%d", &mark);

    total += mark;

}

avg = total / 5.0;

grade = (avg >= 90) ? 'A' : (avg >= 75) ? 'B' : (avg >= 50) ?
'C' : 'F';

printf("\n---      Student      Report      ---
\nName: %s\nRoll: %s\nTotal: %d\nAverage: %.2f\nGrade
: %c\n",

    name, roll, total, avg, grade);

return 0;

}
```

Applications & Case Studies

- **Software Engineering:** Decomposing applications into modules (e.g., login, database, UI).

- **Data Science:** Breaking tasks into data collection, preprocessing, model training, evaluation.
- **Case Study:** ATM System – decomposed into authentication, transaction processing, balance inquiry, withdrawal, deposit.

Comparisons / Notes / Tips

- **Note:** Each subtask should be independent but contribute to solving the main problem.
- **Tip:** Stop decomposition when a subtask becomes simple enough to be implemented as a function.
- **Comparison:** Decomposition in computing is similar to **work breakdown structure (WBS)** in project management.

3.4 SOFTWARE DESIGN CONCEPTS

Software design is the process of **transforming problem requirements into a structured solution** that can be implemented in code. It focuses on how the system will work internally, how components interact, and how to ensure reliability, efficiency, and maintainability.

Software design concepts provide the **principles and strategies** that guide developers when creating programs. Good design ensures that:

- The solution is **modular** and easy to modify.
 - The system is **efficient and scalable**.
 - Code is **reusable and reliable**.
 - The system can evolve with **changing requirements**.

3.4.2 Key Concepts of Software Design

1. **Abstraction** – Hiding unnecessary details and showing only the essential features.
Example: A Car class might have start() and drive(), while the complex engine workings are hidden.
2. **Modularity** – Dividing the program into **independent modules**.
Example: Library management system with modules for issue, return, and catalog search.
3. **Encapsulation** – Binding data and functions that operate on the data into a single unit.
Example: A BankAccount class with private balance and public deposit/withdraw methods.
4. **Information Hiding** – Exposing only what is necessary.
Example: APIs show only function names, not internal implementation.
5. **Reusability** – Writing code that can be reused across projects.
Example: Standard Python libraries like math and os.
6. **Scalability** – Ability of software to handle growing input, users, or complexity.
7. **Maintainability** – Ease of fixing bugs or adding features without breaking existing code.

3.4.5 Step-by-Step Method for Designing Software

1. **Requirement Analysis** – Understand what the system should do.
2. **System Design** – Decide on the architecture (modules, data flow, components).

3. **Detailed Design** – Describe algorithms, data structures, and module interactions.
4. **Implementation Plan** – Assign coding tasks and plan integration.
5. **Testing & Verification** – Ensure each module meets requirements.

Illustration / Example

Problem: Online Food Ordering System

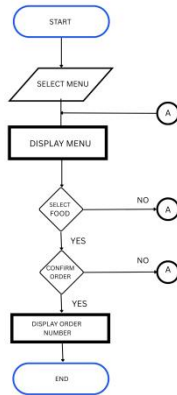
- **Requirement:** Customers should order food online.
- **Decomposition into modules:**
 - Customer Registration
 - Menu Display
 - Order Placement
 - Payment Processing
 - Delivery Tracking
- **Interaction:** Each module communicates with a database and user interface.

Pseudocode

Algorithm FoodOrderingSystem

1. Input user login details
 2. Display menu items
 3. User selects food items
 4. Add items to cart
 5. Process payment
 6. Confirm order
 7. Update delivery status
- End Algorithm

Flowchart Description



Program (Mini Example: Modular Design)

```
#include <stdio.h>
```

```
void display_menu() {  
    printf("1. Pizza - $10\n");  
    printf("2. Burger - $5\n");  
    printf("3. Pasta - $8\n");  
}
```

```
int take_order(char *item) {  
    int choice;
```

```
printf("Enter your choice (1-3): ");
```

```
scanf("%d", &choice);
```

```
switch (choice) {
```

```
    case 1:
```

```
        sprintf(item, "Pizza");
```

```
        return 10;
```

```
    case 2:
```

```
        sprintf(item, "Burger");
```

```
        return 5;
```

```
    case 3:
```

```
        sprintf(item, "Pasta");
```

```
        return 8;
```

```
    default:
```

```
        return 0; // invalid choice
```

```
}
```

```
}

int process_payment(int amount) {

    int paid;

    printf("Amount to pay: $%d\n", amount);

    printf("Enter payment: ");

    scanf("%d", &paid);

    if (paid >= amount) {

        printf("Payment Successful\n");

        return 1; // success

    } else {

        printf("Payment Failed\n");

        return 0; // fail

    }

}

int main() {

    char item[20];
```

```
int cost;

display_menu();

cost = take_order(item);

if (cost > 0) {

    if (process_payment(cost)) {

        printf("Order placed successfully for %s\n", item);

    }

} else {

    printf("Invalid choice.\n");

}

return 0;

}
```

Applications & Case Studies

- **Application in Software Industry:**
 - Used in **banking systems, hospital management, e-commerce websites.**
 - Modular designs make cloud systems like **AWS and Google Cloud** scalable.
- **Case Study: ATM Software Design**

- Modules: Authentication, Transaction Processing, Cash Dispensing.
- Modularity helps in upgrading only one module (e.g., transaction limits) without affecting others.

Comparisons / Notes / Tips

- **Note:** Poor design leads to high maintenance costs.
- **Tip:** Always use modular, reusable code instead of monolithic programs.
- **Comparison:**
 - Procedural Design → Step-by-step functions.
 - Object-Oriented Design → Classes and objects encapsulating data + behavior.

3.5 ALGORITHM

3.5.1 CONCEPT OF AN ALGORITHM

An **algorithm** is a **finite, step-by-step sequence of instructions** that provides a solution to a well-defined problem. In computer science, algorithms form the foundation of all programming — every task a computer performs, from sorting numbers to driving a car autonomously, is guided by algorithms.

Key properties of algorithms:

1. **Finiteness** – The algorithm must terminate after a finite number of steps.
2. **Definiteness** – Each step should be precisely defined.
3. **Input** – The algorithm may have zero or more inputs.
4. **Output** – At least one output must be produced.

5. **Effectiveness** – Each step should be basic enough to be carried out in practice.

Step-by-Step Method for Algorithm Development

1. **Understand the Problem** – Identify the input and expected output.
2. **Plan the Approach** – Think logically about how to solve it.
3. **Divide into Steps** – Break into a sequence of clear instructions.
4. **Check Completeness** – Ensure all cases are handled.
5. **Write Algorithm/Pseudocode** – Express in simple structured form.
6. **Test the Algorithm** – Try with sample data to verify correctness.

3.5.2 ALGORITHM REPRESENTATION

Explanation

Algorithms can be represented in multiple forms:

1. **Natural Language** – Using plain English.
 - Example: “Add two numbers and print result.”
2. **Pseudocode** – Structured English resembling programming.
3. **Flowcharts** – Visual representation using symbols (ovals, rectangles, diamonds).
4. **Code Implementation** – Actual execution in a programming language.

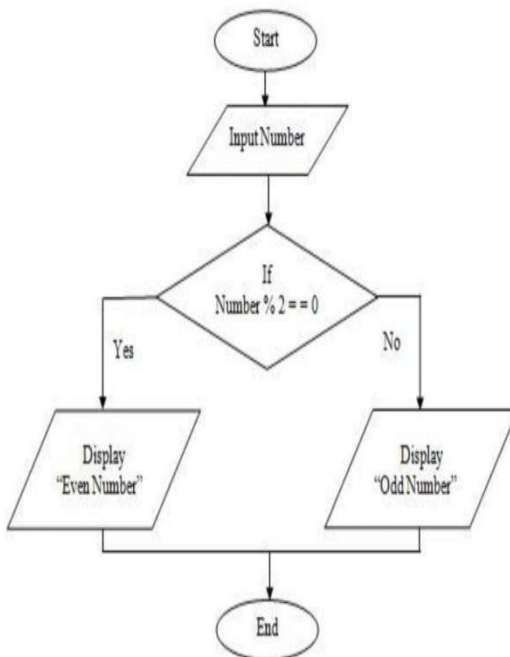
Example

Problem: Check if a number is even or odd.

Pseudocode:

1. Input number n
2. If $n \bmod 2 = 0$ then print "Even"
3. Else print "Odd"

Flowchart Description:



Program:

```
#include <stdio.h>

int main() {

    int n;

    printf("Enter a number: ");

    scanf("%d", &n);

    if (n % 2 == 0) {

        printf("Even\n");

    } else {

        printf("Odd\n");

    }

    return 0;

}
```

3.5.3 ALGORITHM DISCOVERY

Explanation

Algorithm discovery is the process of **finding or creating new algorithms** to solve problems efficiently. It requires **logical thinking, creativity, and optimization skills**.

Methods for discovering algorithms:

1. **Brute Force Method** – Try all possible solutions (simple but inefficient).
2. **Greedy Method** – Build solution step by step, choosing the best option at each step.
3. **Divide and Conquer** – Break problem into smaller subproblems (e.g., Merge Sort).
4. **Dynamic Programming** – Store solutions of subproblems to avoid repetition.
5. **Heuristic / AI Methods** – Use intelligent guesses for complex problems.

Example Problem: Finding Prime Numbers up to n

- **Brute Force Algorithm:** Check each number from 2 to n by dividing with all numbers $< n$.
- **Optimized Algorithm (Sieve of Eratosthenes):** Eliminate multiples of known primes.

C Program (Optimized):

```
#include <stdio.h>
```

```
#include <stdbool.h>

void sieve_of_eratosthenes(int n) {

    bool primes[n+1];

    int i, p;

    // Initialize all entries as true

    for (i = 0; i <= n; i++) {

        primes[i] = true;

    }

    p = 2;

    while (p * p <= n) {

        if (primes[p]) {

            // Mark multiples of p as not prime

            for (i = p * p; i <= n; i += p) {

                primes[i] = false;

            }

        }

    }

}
```

```
    p++;  
  
}  
  
// Print all prime numbers  
for (i = 2; i <= n; i++) {  
    if (primes[i]) {  
        printf("%d ", i);  
    }  
}  
  
printf("\n");  
  
}  
  
int main() {  
    int n = 50;  
  
    sieve_of_eratosthenes(n);  
  
    return 0;  
}
```

Applications & Case Studies

- **Case Study 1:** Google Search uses algorithms for page ranking (PageRank).
- **Case Study 2:** Cryptography uses algorithms for secure data encryption.
- **Case Study 3:** Navigation apps (Google Maps) use shortest path algorithms (Dijkstra's).

Comparisons / Notes / Tips

- **Note:** One problem may have multiple algorithms — choose the most efficient.
- **Tip:** Always analyze algorithm complexity (time & space).
- **Comparison:**
 - Brute Force → Easy but slow.
 - Optimized Algorithms → Complex but efficient.

3.6 ALGORITHMIC STRUCTURES

Concept Explanation

Algorithmic structures are the **basic building blocks** used to control the flow of an algorithm or program. They define **how instructions are executed** — sequentially, conditionally, repeatedly, or recursively.

The main categories of algorithmic structures include:

1. **Sequential Structure** – Instructions executed one after another.

2. **Conditional (Decision) Structure** – Execution depends on conditions.
3. **Iterative (Looping) Structure** – Repeating a block of instructions until a condition is met.
4. **Recursive Structure** – A function calls itself to solve subproblems.

In this section, we focus on **Iterative** and **Recursive** structures.

3.6.1 ITERATIVE STRUCTURES

Concept Explanation

Iteration means **repeating a set of instructions** multiple times until a condition is satisfied. This is implemented using **loops** in programming.

Types of loops in Python:

1. **for loop** – Repeats for a fixed number of times.
2. **while loop** – Repeats as long as a condition is true.
3. **nested loops** – A loop inside another loop.

Step-by-Step Method

1. Identify the part of the problem that repeats.
2. Determine the **loop variable** and **termination condition**.
3. Write the instructions inside the loop.
4. Update the loop variable (if needed).
5. Test the loop with edge cases.

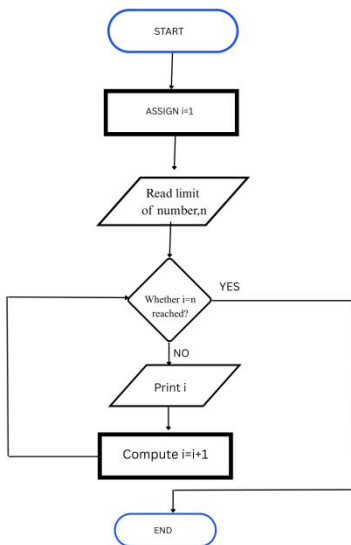
Illustration / Example

Problem: Print the first 5 natural numbers.

- **Pseudocode:**

1. Set $i = 1$
2. While $i \leq 5$:
 Print i
 $i = i + 1$

Flowchart :



Program

```
#include <stdio.h>
```

```
int main() {  
    int i;  
  
    // Using while loop  
  
    i = 1;  
  
    while (i <= 5) {  
        printf("%d\n", i);  
        i++;  
    }  
  
    printf("\n");  
  
    // Using for loop  
  
    for (i = 1; i <= 5; i++) {  
        printf("%d\n", i);  
    }  
  
    return 0;  
}
```

Applications

- Generating series (Fibonacci, primes).
- Searching & sorting algorithms.
- Repetitive tasks like file processing, simulations, animations.

Notes & Tips

- Infinite loops occur if the termination condition is not met.
- For definite repetition → use for.
- For indefinite repetition → use while.

3.6.2 RECURSIVE STRUCTURES

Concept Explanation

Recursion is a technique where a function **calls itself** to solve smaller instances of a problem. It is widely used for problems that can be broken down into similar subproblems.

Properties of recursion:

1. **Base Case** – Condition that stops recursion.
2. **Recursive Case** – Function calls itself with smaller input.

Step-by-Step Method

1. Define the problem in terms of smaller subproblems.
2. Identify the base case (simplest form).
3. Define the recursive call to handle reduced problems.
4. Ensure termination is guaranteed.

Illustration / Example

Problem: Compute factorial of a number ($n!$).

- **Pseudocode:**

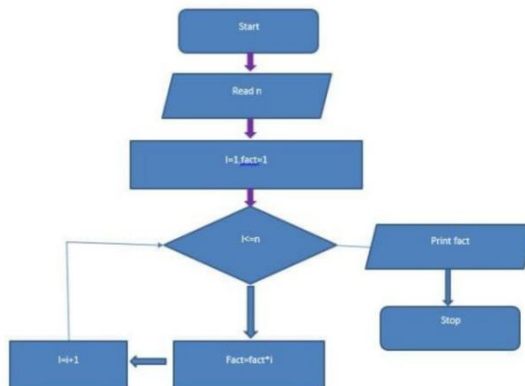
Algorithm Factorial(n)

1. If $n == 0$ then return 1

2. Else return $n * \text{Factorial}(n-1)$

End Algorithm

- **Flowchart :**



Program

```
#include <stdio.h>
```

```
// Recursive function for factorial
```

```
int factorial(int n) {
```

```
    if (n == 0)
```

```
        return 1;
```

```
    else
```

```
        return n * factorial(n - 1);
```

```
}
```

```
int main() {
```

```
    int n = 5;
```

```
    printf("Factorial of %d: %d\n", n, factorial(n));
```

```
    return 0;
```

```
}
```

Applications of Recursion

- **Mathematical Problems:** Factorial, Fibonacci sequence, GCD.
- **Data Structures:** Tree traversals, graph search (DFS).
- **Algorithm Design:** Divide and Conquer methods (Merge Sort, Quick Sort).

Case Study

Tower of Hanoi Problem – Classic recursive problem.

- Move n disks from source $\rightarrow$ destination using an auxiliary peg.
- Recursive solution reduces n -disk problem to $(n-1)$ -disk problem.

Comparisons / Notes / Tips

- **Iteration vs Recursion:**
 - Iteration uses loops, recursion uses function calls.
 - Iteration is usually faster and memory efficient.
 - Recursion is more elegant for problems with self-similar structure (trees, graphs).
- **Tip:** Always define a base case, or recursion will lead to infinite calls (stack overflow).

3.7 EFFICIENCY AND CORRECTNESS OF ALGORITHMS

Concept Explanation

When we design an algorithm, it is not enough for it to just solve the problem—it must also do so **efficiently** and **correctly**.

- **Efficiency** deals with how much **time** and **memory** an algorithm consumes while solving a problem.
- **Correctness** ensures that the algorithm **always produces the right output** for all valid inputs.

Evaluating efficiency and correctness helps in selecting the **best algorithm** among multiple options.

3.7.1 EFFICIENCY OF ALGORITHMS

Definition

Efficiency refers to the **performance** of an algorithm in terms of:

1. **Time Complexity** – How execution time grows with input size.
2. **Space Complexity** – How much memory the algorithm uses.

Step-by-Step Method to Analyze Efficiency

1. **Identify the input size (n):**
Example – number of elements in an array.
2. **Count basic operations:**
Example – comparisons in searching, swaps in sorting.
3. **Express operations as a function of n.**
4. **Use Big-O, Big-Ω, and Big-Θ notations** to represent efficiency.

Illustrations / Examples

Example 1: Linear Search

- Search for an element in an array of size n.
- Worst case → element is at the end or not present.
- Time complexity = **O(n)**.

Example 2: Binary Search

- Requires sorted array.
- Repeatedly divides the array into halves.
- Time complexity = **$O(\log n)$** .

✓ **Binary search is more efficient** than linear search for large datasets.

Pseudocode Example

Linear Search:

Algorithm LinearSearch(A, n, key)

1. For $i = 0$ to $n-1$ do
2. If $A[i] == \text{key}$ then return i
3. End For
4. Return -1

End Algorithm

Binary Search:

Algorithm BinarySearch(A, low, high, key)

1. If $\text{low} > \text{high}$ then return -1
2. $\text{mid} = (\text{low} + \text{high}) / 2$

3. If $A[mid] == key$ then return mid
4. Else if $A[mid] > key$ then return $\text{BinarySearch}(A, \text{low}, \text{mid}-1, \text{key})$
5. Else return $\text{BinarySearch}(A, \text{mid}+1, \text{high}, \text{key})$

End Algorithm

C Implementation

```
#include <stdio.h>
```

```
// Linear Search Function
```

```
int linear_search(int arr[], int n, int key) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (arr[i] == key)
```

```
            return i; // return index
```

```
    }
```

```
    return -1; // not found
```

```
}
```

```
// Binary Search Function
```

```
int binary_search(int arr[], int n, int key) {
```

```
int low = 0, high = n - 1, mid;

while (low <= high) {

    mid = (low + high) / 2;

    if (arr[mid] == key)

        return mid;

    else if (arr[mid] < key)

        low = mid + 1;

    else

        high = mid - 1;

}

return -1; // not found

}

int main() {

    int arr[] = {10, 20, 30, 40, 50};

    int n = sizeof(arr) / sizeof(arr[0]);

    int key = 40;
```

```
int lin_result = linear_search(arr, n, key);

int bin_result = binary_search(arr, n, key);

if (lin_result != -1)

    printf("Linear Search (%d): Found at index %d\n", key,
lin_result);

else

    printf("Linear Search (%d): Not Found\n", key);

if (bin_result != -1)

    printf("Binary Search (%d): Found at index %d\n", key,
bin_result);

else

    printf("Binary Search (%d): Not Found\n", key);

return 0;

}
```

Applications

- **Time efficiency** critical in real-time systems (traffic control, stock trading).
- **Space efficiency** critical in embedded systems, mobile devices.

- Used in selecting sorting/searching techniques depending on dataset size.

3.7.2 CORRECTNESS OF ALGORITHMS

Definition

An algorithm is correct if it produces the **expected output** for every valid input.

Correctness can be classified into:

1. **Partial Correctness** – If the algorithm terminates, it gives the correct result.
2. **Total Correctness** – The algorithm terminates and always gives the correct result.

Step-by-Step Method to Prove Correctness

1. **Understand the problem specification** (input, output).
2. **Establish invariants** – conditions that remain true during execution.
3. **Show base case correctness.**
4. **Show correctness for each step.**
5. **Show termination condition.**

Illustration / Example

Problem: Checking if an array is sorted.

- **Loop Invariant:** At the end of each iteration, the first i elements are in sorted order.

- **Termination:** At the end of loop, if no violations are found, the array is sorted.

Pseudocode

Algorithm CheckSorted(A, n)

1. For $i = 0$ to $n-2$ do
2. If $A[i] > A[i+1]$ then return False
3. End For
4. Return True

End Algorithm

Program

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
// Function to check if array is sorted in ascending order
```

```
bool is_sorted(int arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        if (arr[i] > arr[i + 1])
```

```
            return false;
```

```
}

return true;

}

int main() {

    int arr1[] = {1, 2, 3, 4, 5};

    int arr2[] = {5, 3, 2, 1};


    int n1 = sizeof(arr1) / sizeof(arr1[0]);

    int n2 = sizeof(arr2) / sizeof(arr2[0]);

    if (is_sorted(arr1, n1))

        printf("arr1 is sorted: True\n");

    else

        printf("arr1 is sorted: False\n");

    if (is_sorted(arr2, n2))

        printf("arr2 is sorted: True\n");

    else
```

```
printf("arr2 is sorted: False\n");  
  
return 0;  
  
}
```

Applications

- Verification of **cryptographic algorithms** (must always be correct).
- **Compiler correctness** – ensuring generated code is accurate.
- **Financial systems** – correctness ensures no calculation errors.

Comparison of Efficiency vs Correctness

Feature	Efficiency (Time/Space)	Correctness (Accuracy)
Concern	Performance	Reliability
Measured in	Big-O notation	Logical proofs, testing
Trade-off	Faster code may be complex	Correct code may be slower
Example	Merge Sort vs Bubble Sort	Checking sorted arrays

Notes & Tips

- A correct algorithm is **useless if inefficient** (too slow for real-world use).
- An efficient algorithm is **dangerous if incorrect** (wrong outputs).
- Always aim for a **balance** – correctness first, then optimize efficiency.

3.8 IMPLEMENTATION OF ALGORITHMS

Concept Explanation

Once an algorithm is designed, the next step is to **implement** it in a programming language so that it can be executed on a computer. Implementation translates the **logical design (pseudocode/flowchart)** into **executable code**.

- **Algorithm** = Logical steps (language-independent).
- **Implementation** = Actual code in Python, C, Java, etc.

Implementation is a crucial phase because:

1. It determines whether the designed algorithm works in practice.
2. It exposes hidden flaws in logic.
3. It allows testing efficiency and correctness.

Step-by-Step Method for Implementation

1. **Define the Problem Clearly**
 - Understand input, output, and constraints.

- Example: “Find the sum of first n natural numbers.”
- 2. **Design the Algorithm**
 - Write pseudocode or draw a flowchart.
- 3. **Select the Programming Language**
 - Choose Python for simplicity, C for performance, Java for portability, etc.
- 4. **Translate Algorithm into Code**
 - Implement step-by-step logic using syntax.
- 5. **Test with Sample Data**
 - Try small cases to check correctness.
- 6. **Debug Errors**
 - Fix syntax, logical, and runtime errors.
- 7. **Optimize the Implementation**
 - Improve efficiency if needed.

Illustrations / Examples

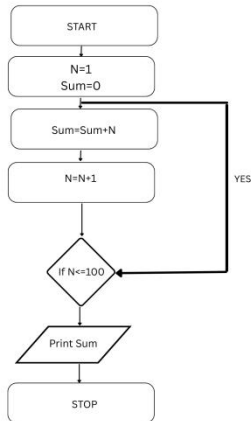
Example 1: Summation of Numbers

Algorithm (Pseudocode):

Algorithm SumNumbers(n)

1. $\text{sum} \leftarrow 0$
 2. For $i = 1$ to n do
 3. $\text{sum} \leftarrow \text{sum} + i$
 4. End For
 5. Return sum
- End Algorithm

Flowchart:



C Implementation:

```
#include <stdio.h>
```

```
// Function to calculate sum of first n numbers
```

```
int sum_numbers(int n) {  
    int total = 0;  
    for (int i = 1; i <= n; i++) {  
        total += i;  
    }  
    return total;  
}
```

```
int main() {  
    int n = 10;  
    printf("Sum of first %d numbers: %d\n", n, sum_numbers(n));  
    return 0;  
}
```

Example 2: Factorial of a Number**Algorithm (Pseudocode):**

Algorithm Factorial(n)

1. fact $\leftarrow$ 1
 2. For i = 1 to n do
 3. fact $\leftarrow$ fact * i
 4. End For
 5. Return fact
- End Algorithm

C Implementation:

```
#include <stdio.h>
```

```
// Function to calculate factorial iteratively
```

```
int factorial(int n) {  
  
    int fact = 1;  
  
    for (int i = 1; i <= n; i++) {  
  
        fact *= i;  
  
    }  
  
    return fact;  
  
}
```

```
int main() {  
  
    int n = 5;  
  
    printf("Factorial of %d: %d\n", n, factorial(n));  
  
    return 0;  
  
}
```

Example 3: Fibonacci Sequence

Algorithm (Pseudocode):

Algorithm Fibonacci(n)

1. $a \leftarrow 0, b \leftarrow 1$
2. Print a, b
3. For $i = 2$ to n do
4. $c \leftarrow a + b$
5. Print c
6. $a \leftarrow b$
7. $b \leftarrow c$
8. End For

End Algorithm

C Implementation:

```
#include <stdio.h>
```

```
void fibonacci(int n) {  
    int a = 0, b = 1, c;
```

```
if (n >= 1) printf("%d ", a);
if (n >= 2) printf("%d ", b);

for (int i = 2; i < n; i++) {
    c = a + b;
    printf("%d ", c);
    a = b;
    b = c;
}

printf("\n");
}

int main() {
    int n = 10;
    fibonacci(n);
    return 0;
}
```

Applications of Algorithm Implementation

- **Scientific Computing:** Implementing mathematical models.
- **Engineering:** Simulations, automation, control systems.
- **Data Science:** Algorithms for sorting, searching, machine learning.
- **Cybersecurity:** Implementing encryption algorithms.
- **Daily Applications:** Banking transactions, online shopping, route-finding in maps.

Comparisons, Notes, and Tips

Aspect	Algorithm Design	Implementation
Nature	Theoretical	Practical
Language	Pseudocode, flowcharts	Python, Java, C++
Purpose	To define logic	To execute logic
Errors Found	Logical errors	Syntax + runtime + logical errors
Example	Bubble Sort Algorithm	Python code for Bubble Sort

Tips for Implementation:

1. Start with **small test cases**.
2. Use **print statements** for debugging.
3. Write **modular code** (functions).
4. Compare outputs with **manual calculations**.
5. Use **comments** to explain steps.

Case Study: Sorting Student Marks

Problem: Given marks of students, sort them in ascending order.

Algorithm (Pseudocode):

Algorithm SortMarks(M, n)

1. For $i = 0$ to $n-1$ do
2. For $j = 0$ to $n-i-1$ do
3. If $M[j] > M[j+1]$ then
4. Swap $M[j]$, $M[j+1]$
5. End For

6. End For
7. Return M
End Algorithm

C Implementation:

```
#include <stdio.h>

// Function to perform bubble sort

void bubble_sort(int arr[], int n) {

    int i, j, temp;

    for (i = 0; i < n - 1; i++) {

        for (j = 0; j < n - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {

                // Swap arr[j] and arr[j+1]

                temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

            }

        }

    }

}
```

```
    }  
}  
  
// Function to print array  
void print_array(int arr[], int n) {  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}  
  
int main() {  
    int marks[] = {56, 89, 34, 90, 12};  
    int n = sizeof(marks) / sizeof(marks[0]);  
    bubble_sort(marks, n);  
    printf("Sorted Marks: ");  
    print_array(marks, n);  
    return 0;  
}
```

}

3.9 FUNDAMENTAL ALGORITHMS

3.9.1 EXCHANGING VALUES OF TWO VARIABLES

Concept:

Often in programming, we need to swap the values of two variables. For example, if $A = 5$ and $B = 10$, after swapping, $A = 10$ and $B = 5$. This is essential in sorting algorithms, memory management, and data manipulation.

Methods:

1. Using a temporary variable (safe and standard).
2. Without temporary variable (using arithmetic or XOR operation).

Pseudocode (with temporary variable):

START

Input A, B

TEMP $\leftarrow$ A

A $\leftarrow$ B

B $\leftarrow$ TEMP

Print A, B

STOP

Example:

- Input: A = 5, B = 10
- Output: A = 10, B = 5

Practical Use:

- Bubble sort and other sorting algorithms.
- Data shuffling.

3.9.2 COUNTING

Concept:

Counting is the process of iterating through a sequence of numbers in order. It forms the basis of iteration and loop constructs. For example, printing numbers 1 to 10.

Logic:

- Start with a counter variable.
- Repeat until a condition is met.
- Increment the counter.

Pseudocode:

START

Input N

SET $i = 1$

WHILE $i \leq N$

 Print i

$i \leftarrow i + 1$

END WHILE

STOP

Example:

- Input: $N = 5$
- Output: 1 2 3 4 5

Practical Use:

- Looping through datasets.
- Array traversal.
- Generating sequence numbers.

3.9.3 SUMMATION OF A SET OF NUMBERS

Concept:

Summation refers to adding a sequence of numbers. For example, calculating the total marks of students or total sales in a shop.

Logic:

- Initialize $\text{sum} = 0$.
- Add each number to sum.

- Print final result.

Pseudocode:

START

Input N

SET sum = 0

FOR i = 1 to N

 Input number

 sum = sum + number

END FOR

Print sum

STOP

Example:

- Input: N = 5, numbers = 2, 4, 6, 8, 10
- Output: Sum = 30

Practical Use:

- Statistical calculations.
- Budgeting and accounting.
- Data aggregation in machine learning.

3.9.4 FACTORIAL COMPUTATION

Concept:

Factorial of a number n is the product of all positive integers up to n .

- Formula: $n! = n \times (n-1) \times (n-2) \dots \times 1$
- Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Methods:

1. **Iterative method (loop).**
2. **Recursive method (function calls itself).**

Pseudocode (Iterative):

START

Input n

SET $\text{fact} = 1$

FOR $i = 1$ to n

$\text{fact} = \text{fact} * i$

END FOR

Print fact

STOP

Pseudocode (Recursive):

FUNCTION Factorial(n)

IF $n == 0$ OR $n == 1$ THEN

RETURN 1

ELSE

RETURN $n * \text{Factorial}(n-1)$

END FUNCTION

Practical Use:

- Permutations and combinations.
- Probability and statistics.
- Algorithms in mathematics and computer science.

3.9.5 GENERATION OF FIBONACCI SEQUENCE

Concept:

The Fibonacci sequence is a series where each number is the sum of the two preceding ones.

- Formula: $F_n = F_{n-1} + F_{n-2}$
- Example: 0, 1, 1, 2, 3, 5, 8, 13...

Logic:

- Start with two numbers (0 and 1).
- Keep adding them to generate the next number.

Pseudocode:

START

Input N

SET $a = 0$, $b = 1$

Print a, b

FOR $i = 3$ to N

$c = a + b$

 Print c

$a = b$

$b = c$

END FOR

STOP

Practical Use:

- Recursive programming.
- Growth models in nature (like rabbit population).
- Used in computer algorithms (e.g., Fibonacci search).

3.9.6 REVERSING THE DIGITS OF AN INTEGER

Concept:

Reversing digits means rearranging the digits of a number in reverse order.

- Example: Input 1234 $\rightarrow$ Output 4321.

Logic:

- Extract the last digit using modulus %.
- Add it to the reversed number.
- Remove last digit by dividing by 10.
- Repeat until number becomes 0.

Pseudocode:

START

Input num

SET rev = 0

WHILE num > 0

 digit = num % 10

 rev = rev * 10 + digit

 num = num / 10

END WHILE

Print rev

STOP

Practical Use:

- Checking for palindrome numbers.
- Data encryption and manipulation.
- Digital signal processing.

3.9.7 BASE CONVERSION

Concept:

Base conversion means changing numbers from one number system to another.

- Decimal (Base 10) $\rightarrow$ Binary (Base 2), Octal (Base 8), Hexadecimal (Base 16).
- Example: Decimal 10 $\rightarrow$ Binary 1010 $\rightarrow$ Octal 12 $\rightarrow$ Hexadecimal A.

Logic (Decimal to Binary):

- Divide number by 2.
- Record remainder.
- Repeat until quotient = 0.
- Read remainders in reverse order.

Pseudocode:

START

Input decimal_number

SET binary = ""

WHILE decimal_number > 0

 remainder = decimal_number % 2

 binary = remainder + binary

 decimal_number = decimal_number / 2

END WHILE

Print binary

STOP

Practical Use:

- Computer systems use binary, octal, hexadecimal representations.
- Low-level programming and networking.
- Cryptography and data encoding.

3.10 IMPORTANT TWO MARKS

1) **Define an algorithm.**

An algorithm is a step-by-step procedure or set of rules for solving a problem in a finite number of steps.

2) **What is problem decomposition?**

Problem decomposition is dividing a complex problem into smaller sub-problems that are easier to solve.

3) **What is logical reasoning?**

Logical reasoning is the process of applying structured thinking to derive conclusions or solutions.

4) **List two characteristics of a good algorithm.**

- Finiteness (must terminate after a finite number of steps).
- Definiteness (each step must be clear and unambiguous).

5) **Differentiate between iterative and recursive structures.**

- Iterative structure uses loops (for, while).
- Recursive structure involves a function calling itself.

6) **Define correctness of an algorithm.**

An algorithm is correct if it produces the expected output for all valid inputs.

7) **What is efficiency of an algorithm?**

Efficiency refers to how effectively an algorithm uses resources such as time and memory.

8) **Write the steps to exchange two variables?**

temp = a

a = b

b = temp

9) **What are the first two numbers of the Fibonacci sequence?**

The first two numbers are 0 and 1.